# UNIVERSIDAD PRIVADA DOMINGO SAVIO

**ACTIVIDAD 1: DESARROLLO DE UNA APLICACIÓN DE CATÁLOGO DE PRODUCTOS EN PHP 8 CON PROGRAMACIÓN ORIENTADA A OBJETOS Y PERSISTENCIA EN JSON**

**Carrera:** Ingeniería de Sistemas

**Asignatura:** Tecnología Web II

**Docente:** M.Sc. Ing. Jimmy Nataniel Requena Llorentty


**Integrantes:**

Alejandro Fabián Cabrera Flores

María Alejandra Suárez O.

Jade Guissel Flores Coria

Ariel Salas Tococari

Renzo Molina Navia


Santa Cruz de la Sierra - Octubre 2026

# ÍNDICE


1. INTRODUCCIÓN ..... 3
2. PLANTEAMIENTO DEL PROBLEMA ..... 4
3. JUSTIFICACIÓN ..... 4
4. OBJETIVOS ..... 5
   4.1. Objetivo general ..... 5
   4.2. Objetivos específicos ..... 5
5. MARCO TEÓRICO ..... 6
   5.1. PHP 8 y programación del lado del servidor ..... 6
   5.2. Tipos estrictos y funciones tipadas ..... 6
   5.3. Programación orientada a objetos ..... 6
   5.4. Formularios GET y POST ..... 7
   5.5. Persistencia en archivos JSON ..... 7
   5.6. Seguridad en la subida de fotografías ..... 7
   5.7. Pruebas automatizadas ..... 7
6. DESARROLLO DE LA APLICACIÓN ..... 8
   6.1. Entorno de desarrollo ..... 8
   6.2. Organización del proyecto ..... 8
   6.3. Implementación de las clases ..... 9
   6.4. Funciones y procesamiento de datos ..... 10
   6.5. Formularios y validación ..... 10
   6.6. Persistencia en JSON ..... 11
   6.7. Subida y administración de fotografías ..... 12
   6.8. Pruebas y verificación ..... 13
7. REFLEXIÓN CRÍTICA ..... 14

8. CONCLUSIONES ..... 15

9. RECOMENDACIONES ..... 15

## 1. INTRODUCCIÓN

El desarrollo de aplicaciones web requiere el uso de herramientas y técnicas que permitan organizar la información, validar los datos ingresados por los usuarios y mantener la información almacenada de manera persistente. PHP es un lenguaje de programación que permite implementar estas funcionalidades desde el servidor y generar contenido dinámico para el navegador.

El presente trabajo consiste en desarrollar una aplicación de catálogo de productos para una asociación de productores, integrando los conocimientos adquiridos en los cuatro laboratorios de Tecnología Web II. La aplicación busca facilitar el registro, consulta, edición y eliminación de productos, además de permitir la incorporación de fotografías y el almacenamiento de la información mediante archivos JSON.

Para su desarrollo se emplean PHP 8, programación orientada a objetos (POO), funciones tipadas, formularios GET y POST, validación de entradas, manejo de archivos y organización del código mediante include y require. También se contempla el uso de las clases Producto, Categoria y Catalogo, con promoción de propiedades en los constructores.

Finalmente, se realizarán pruebas funcionales para comprobar el comportamiento de la aplicación y se documentarán las evidencias del proceso, incluyendo las comprobaciones automatizadas, la presentación del sistema, el video demostrativo y la bitácora de uso de inteligencia artificial.

## 2. PLANTEAMIENTO DEL PROBLEMA

La administración manual de un catálogo puede generar dificultades al actualizar los precios, registrar nuevos productos, controlar las existencias y mantener las fotografías asociadas a cada registro. Cuando la información se encuentra distribuida o debe modificarse directamente en el código, el mantenimiento se vuelve más complejo y aumenta la posibilidad de cometer errores.

Ante esta situación, se plantea desarrollar una aplicación web que centralice las operaciones básicas de un catálogo de productos. La solución debe permitir que los datos se registren mediante formularios, se validen antes de guardarse y permanezcan disponibles después de cerrar y volver a abrir la aplicación.

Asimismo, es necesario evitar que las entradas incorrectas o los archivos no válidos comprometan el funcionamiento del sistema. Por ello, se incorporan mecanismos de validación, saneamiento, control de rutas y comprobación del contenido real de las imágenes.

## 3. JUSTIFICACIÓN

La realización de este proyecto permite aplicar de manera práctica los conocimientos adquiridos durante los cuatro laboratorios, relacionando los fundamentos del lenguaje PHP con el desarrollo de una aplicación web funcional.

El uso de programación orientada a objetos facilita la organización de los productos, las categorías y las operaciones del catálogo. Por su parte, el

almacenamiento en JSON representa una alternativa sencilla para conservar los registros sin necesidad de configurar un sistema gestor de bases de datos para esta actividad.

La validación de formularios y la gestión segura de fotografías contribuyen a mejorar la confiabilidad de la aplicación. Además, las pruebas automatizadas permiten identificar errores y verificar si las funciones implementadas cumplen los requisitos establecidos.

El proyecto también fortalece las habilidades de análisis, resolución de problemas, documentación técnica y uso responsable de herramientas de inteligencia artificial como apoyo al aprendizaje.

## 4. OBJETIVOS

### 4.1. Objetivo general

Desarrollar una aplicación web de catálogo de productos utilizando PHP 8 y programación orientada a objetos, incorporando formularios validados, gestión segura de fotografías y persistencia de datos en archivos JSON, para comprobar su funcionamiento mediante pruebas y evidencias técnicas.

### 4.2. Objetivos específicos

- Configurar el entorno de desarrollo con PHP 8.4, Visual Studio Code y el servidor web integrado de PHP.
- Aplicar tipos estrictos, funciones tipadas, estructuras de control, arreglos y operaciones con cadenas.
- Organizar el código mediante las clases Producto, Categoria y Catalogo, utilizando promoción de propiedades en los constructores.
- Implementar formularios GET y POST con validación de datos y manejo adecuado de errores.
- Incorporar operaciones para registrar, consultar, editar y eliminar productos.
- Implementar la lectura y escritura de información en archivos JSON para conservar los registros.
- Establecer controles de seguridad para la subida, almacenamiento y entrega de fotografías de productos.
- Ejecutar las pruebas automatizadas y documentar los resultados obtenidos.

- Completar la bitácora de inteligencia artificial y preparar las evidencias solicitadas para la entrega.

## 5. MARCO TEÓRICO

### 5.1. PHP 8 y programación del lado del servidor

PHP es un lenguaje utilizado para desarrollar aplicaciones del lado del servidor. Permite recibir solicitudes del navegador, procesar formularios, realizar operaciones con datos y generar páginas HTML dinámicas.

En este proyecto se utiliza PHP para implementar la lógica del catálogo, validar los datos ingresados y controlar las operaciones realizadas sobre los productos.

### 5.2. Tipos estrictos y funciones tipadas

La declaración declare(strict_types=1); permite aplicar un control más estricto sobre los tipos escalares en las llamadas a funciones realizadas desde el archivo correspondiente.

Las funciones tipadas especifican los tipos de sus parámetros y, cuando corresponde, el tipo de dato que devuelven. Esto facilita la comprensión del código y permite detectar determinados errores relacionados con el uso incorrecto de los datos.

También se pueden utilizar parámetros nombrados y valores predeterminados para mejorar la claridad de las llamadas a las funciones.

### 5.3. Programación orientada a objetos

La programación orientada a objetos organiza el programa mediante clases y objetos que representan elementos del sistema.

En la aplicación se consideran las siguientes clases:

- **Producto:** representa cada artículo del catálogo y sus atributos, como código, nombre, categoría, precio, stock y fotografía.
- **Categoria:** representa y organiza las categorías utilizadas para clasificar los productos.
- **Catalogo:** administra la colección de productos y las operaciones necesarias para consultarlos, registrarlos, actualizarlos y eliminarlos.

La promoción de propiedades en el constructor permite declarar e inicializar propiedades de una clase directamente en los parámetros del constructor, reduciendo código repetitivo.

### 5.4. Formularios GET y POST

El método GET permite enviar parámetros mediante la URL, por lo que resulta útil para las búsquedas y los filtros del catálogo. El método POST permite enviar información en el cuerpo de la solicitud y se utiliza para operaciones como el registro y la modificación de productos.

La información recibida debe validarse en el servidor antes de utilizarse. La validación comprueba que los datos cumplan los requisitos establecidos, mientras que el saneamiento o la codificación apropiada ayudan a evitar que las entradas se interpreten de manera no deseada.

### 5.5. Persistencia en archivos JSON

JSON es un formato de intercambio de datos que permite representar estructuras mediante objetos y arreglos. PHP proporciona funciones como json_encode() y json_decode() para convertir información entre estructuras de PHP y texto JSON.

La persistencia consiste en guardar los datos de manera que continúen disponibles después de finalizar una ejecución. En el proyecto, los archivos JSON permiten conservar la información del catálogo sin depender de una base de datos externa.

### 5.6. Seguridad en la subida de fotografías

La recepción de archivos requiere controles adicionales porque el nombre, la extensión y el tipo declarado por el navegador no garantizan que el contenido sea realmente una imagen.

Para reducir riesgos, se deben comprobar los errores de subida, el tamaño del archivo, el tipo MIME real y la validez de la imagen. También conviene generar nombres aleatorios, restringir las extensiones permitidas y evitar que los parámetros recibidos permitan acceder a archivos fuera del directorio autorizado.

### 5.7. Pruebas automatizadas

Las pruebas automatizadas permiten ejecutar una serie de comprobaciones para verificar el comportamiento de una aplicación. En los laboratorios se utilizaron verificadores PHP para revisar funciones, formularios, almacenamiento y operaciones sobre archivos.

En el proyecto integrado, los resultados deben comprobarse nuevamente, ya que superar las pruebas individuales no garantiza automáticamente que todos los componentes funcionen correctamente en conjunto.

# 6. DESARROLLO DE LA APLICACIÓN

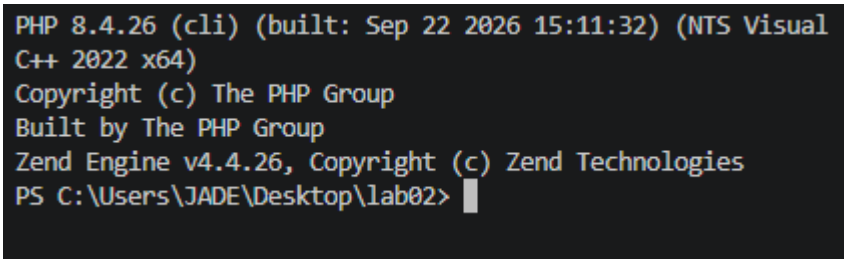
## 6.1. Entorno de desarrollo

Para la implementación se contempla el siguiente entorno:

- Lenguaje: PHP 8.4.
- Editor de código: Visual Studio Code.
- Servidor: servidor web integrado de PHP.
- Almacenamiento: archivos JSON.
- Navegador web: [indicar el navegador utilizado].
- Herramientas de verificación: scripts PHP de pruebas.

**Figura 1**

*Entorno de desarrollo utilizado para el proyecto.*




**Nota.** Elaboración propia.

## 6.2. Organización del proyecto

La aplicación se organiza en archivos y directorios separados para mantener diferenciadas la lógica del sistema, las páginas públicas, las plantillas, los datos y las fotografías.

La estructura definitiva deberá corresponder con los archivos reales del repositorio. Una organización de referencia puede incluir:

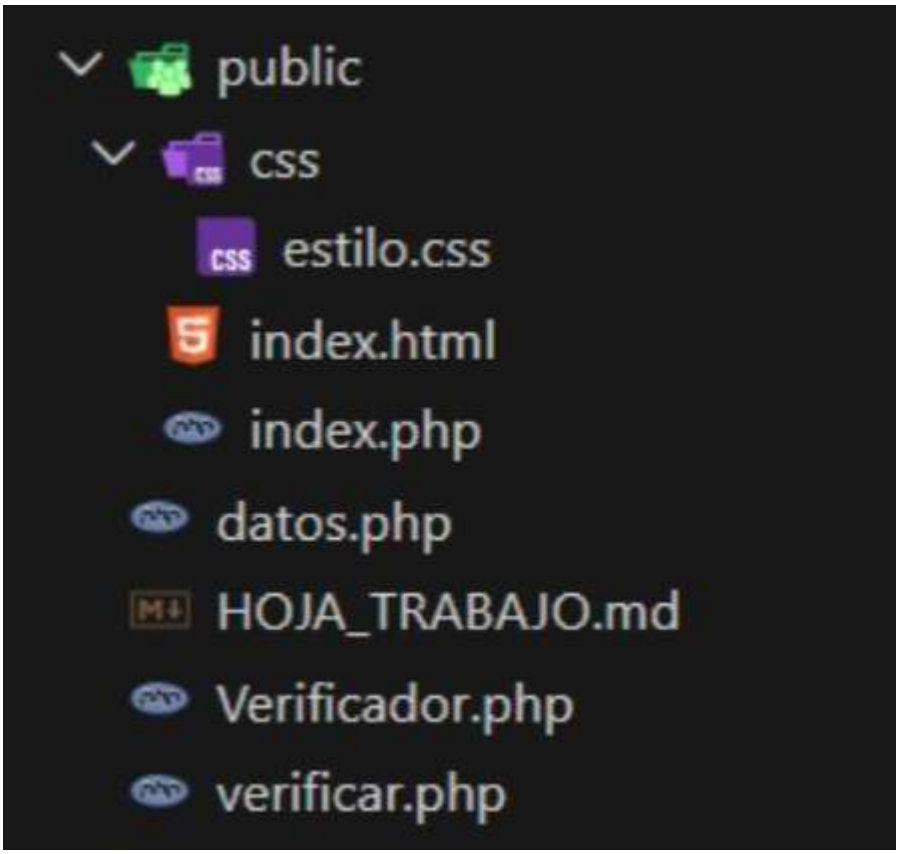
- src/: clases, funciones de validación y lógica del catálogo.
- public/: páginas accesibles desde el navegador, formularios y recursos públicos.
- plantillas/: componentes reutilizables de la interfaz.
- datos/: archivos JSON y almacenamiento de información.
- datos/fotos/: fotografías de los productos, si se utiliza esta ubicación.

- verificar.php o los scripts equivalentes: comprobaciones automatizadas.

Los archivos que contienen lógica interna y datos no deben exponerse directamente al navegador.

**Figura 2**

*Estructura de directorios y archivos de la aplicación.*




**Nota.** Elaboración propia.

### 6.3. Implementación de las clases

La programación orientada a objetos permite representar los elementos principales del catálogo y separar sus responsabilidades.

La clase Producto concentra los datos de cada producto; Categoria representa su clasificación, y Catalogo administra las operaciones sobre la colección. Los constructores permiten inicializar los objetos, utilizando promoción de propiedades cuando corresponda.

### 6.4. Funciones y procesamiento de datos

Se incorporan funciones tipadas para realizar operaciones como el cálculo de descuentos, la clasificación del stock, el filtrado por categoría, la ordenación por precio, la suma del valor del inventario y la validación de códigos.

También se utilizan arreglos y funciones de procesamiento para evitar repetir lógica y facilitar el mantenimiento del código. La declaración de tipos estrictos ayuda a detectar usos incompatibles de los datos, mientras que las funciones para cadenas multibyte permiten trabajar correctamente con caracteres como las tildes y la letra ñ.

**Figura 3**

*Funciones tipadas y procesamiento de la información del inventario.*




**Nota.** Elaboración propia.

### 6.5. Formularios y validación

Los formularios permiten buscar y filtrar productos mediante GET y registrar o modificar información mediante POST.

Antes de guardar los datos, el sistema debe verificar los campos obligatorios, el formato del código, la categoría, el precio, el stock y los demás atributos definidos en el proyecto. Cuando se detectan errores, se deben mostrar mensajes comprensibles y evitar el almacenamiento de registros inválidos.

Cuando corresponda, se utiliza el patrón POST/Redirect/GET para redirigir después de una operación válida y reducir el riesgo de repetir accidentalmente el envío del formulario al actualizar la página.

**Figura** **4**

*Formulario de registro y validación de los datos del producto.*




**Nota.** Elaboración propia.

### 6.6. Persistencia en JSON

La información del catálogo se conserva en archivos JSON. Para ello, la aplicación debe leer los registros existentes, convertirlos a estructuras utilizables por PHP y guardar los cambios después de las operaciones correspondientes.

La codificación debe contemplar los caracteres Unicode y el tratamiento de errores. Asimismo, se deben tomar precauciones al escribir los archivos para reducir el riesgo de escrituras incompletas o simultáneas.

**Figura** **5**

*Persistencia de los productos en formato JSON.*




**Nota.** Elaboración propia.

### 6.7. Subida y administración de fotografías

El sistema incorpora la recepción de fotografías asociadas a los productos. Para mantener un manejo seguro, la validación se realiza en el servidor, comprobando el contenido real del archivo y rechazando los formatos no permitidos o los archivos que superen el tamaño establecido.

Las fotografías deben almacenarse con nombres controlados y entregarse mediante rutas restringidas. Cuando se reemplaza una imagen, también se debe gestionar correctamente la fotografía anterior. Las operaciones de modificación y eliminación se procesan mediante métodos apropiados y no mediante enlaces GET que ejecuten cambios.

**Figura** **6**

*Gestión de fotografías de los productos.*




**Nota.** Elaboración propia.

### 6.8. Pruebas y verificación

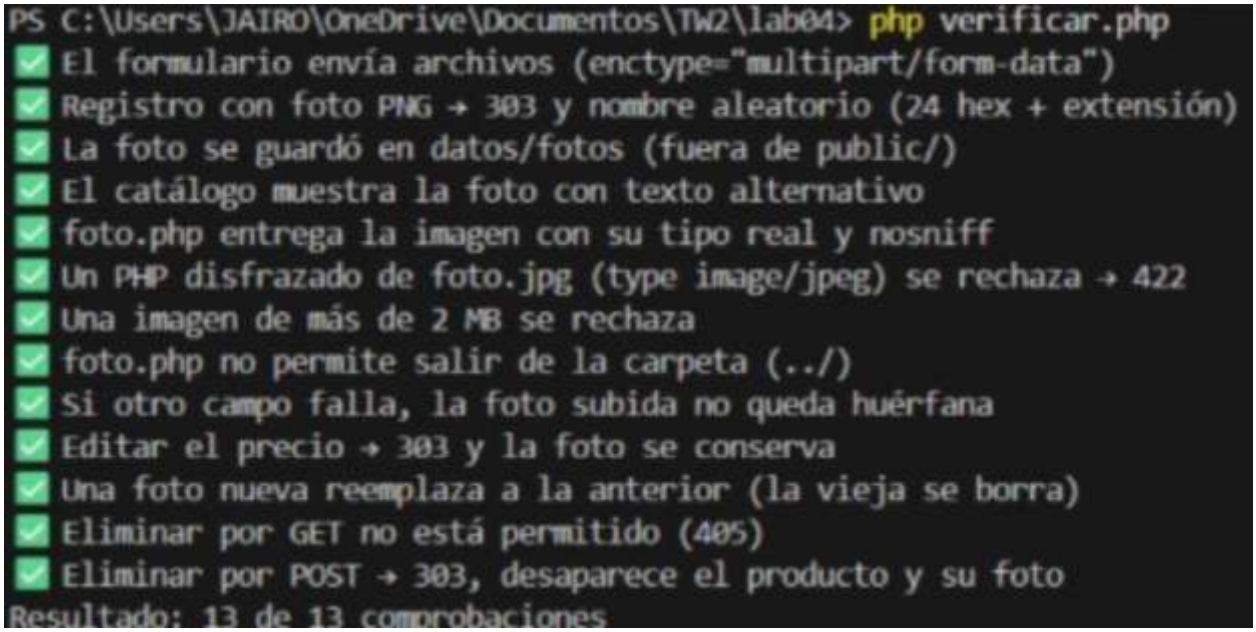
La aplicación se verificará mediante los scripts de prueba disponibles y comprobaciones funcionales desde el navegador.

Entre los aspectos que deben evaluarse se encuentran:

- Funcionamiento de las funciones y cálculos del inventario.
- Validación de los datos de entrada.
- Registro de productos y conservación de la información en JSON.
- Consulta, edición y eliminación de productos.
- Aceptación de imágenes válidas y rechazo de archivos no permitidos.
- Control de rutas y manejo de errores.
- Funcionamiento conjunto de las clases y los demás componentes.

**Figura 7**

*Resultado del verificador automatizado del proyecto integrado.*




**Nota.** Elaboración propia.

No se debe colocar una cantidad de pruebas aprobadas hasta ejecutar el verificador definitivo de la actividad.

## 7. REFLEXIÓN CRÍTICA

El desarrollo de la aplicación permitió relacionar los conceptos estudiados en los laboratorios con un caso práctico. Al principio, las operaciones del catálogo podían resolverse mediante arreglos y funciones; sin embargo, al incorporar formularios, fotografías y persistencia, fue necesario organizar mejor el código y controlar con mayor cuidado los datos recibidos.

Uno de los aspectos importantes fue comprender que una aplicación no debe limitarse a mostrar los productos. También necesita validar la información, conservar los cambios y controlar los archivos que recibe. Las pruebas resultaron útiles para detectar errores y comprobar si las modificaciones realizadas producían el comportamiento esperado.

La inteligencia artificial facilitó la aclaración de algunas dudas, pero sus sugerencias no se consideraron correctas automáticamente. Fue necesario revisar las propuestas, adaptarlas al código del proyecto y comprobar sus resultados. De esta

manera, se utilizó como apoyo al aprendizaje y no como sustituto de la implementación y la verificación personal.

## 8. CONCLUSIONES

1. El proyecto integra los conocimientos de los cuatro laboratorios mediante el desarrollo de una aplicación de catálogo que combina procesamiento de datos, formularios, almacenamiento y administración de archivos. **Alejandro Cabrera Flores.**
2. La programación orientada a objetos permite organizar las responsabilidades del sistema en las clases Producto, Categoria y Catalogo, mientras que las funciones tipadas y los tipos estrictos contribuyen a mantener una lógica más clara. **María Alejandra Suárez O.**
3. La persistencia en JSON permite conservar los registros sin depender de un sistema gestor de bases de datos externo, aunque requiere un manejo adecuado de los errores y de las operaciones de lectura y escritura. **Jade Flores Coria.**
4. La validación de formularios y la comprobación de las fotografías son necesarias para evitar el ingreso de datos incorrectos y reducir los riesgos relacionados con la recepción de archivos. **Ariel Salas Tococari.**
5. La comprobación final de la aplicación debe respaldarse con resultados reales de las pruebas automatizadas, evidencias de funcionamiento y una bitácora que describa el uso de las herramientas de apoyo. **Renzo Molina Navia.**

## 9. RECOMENDACIONES

- Ejecutar nuevamente todos los verificadores después de integrar los cuatro laboratorios.
- Comprobar el funcionamiento de las operaciones principales desde el navegador.
- Revisar que los datos permanezcan guardados después de reiniciar el servidor.
- Verificar que las fotografías se validen en el servidor y que no sea posible acceder a archivos fuera de las rutas autorizadas.
- Mantener separadas la lógica de negocio, las plantillas y los archivos públicos.
- Registrar en la bitácora las consultas reales realizadas a la inteligencia artificial y las comprobaciones efectuadas.

- Revisar el formato del documento, las referencias, la numeración de las figuras y las capturas antes de generar el PDF definitivo.

**BIBLIOGRAFÍA**


PHP Documentation Group. (s. f.). *PHP manual*. PHP. https://www.php.net/manual/es/

PHP Documentation Group. (s. f.). *Classes and objects*. PHP Manual. https://www.php.net/manual/en/language.oop5.php

PHP Documentation Group. (s. f.). *Handling file uploads*. PHP Manual. https://www.php.net/manual/en/features.file-upload.php

PHP Documentation Group. (s. f.). *JSON*. PHP Manual. https://www.php.net/manual/en/book.json.php

PHP Documentation Group. (s. f.). *Function arguments*. PHP Manual. https://www.php.net/manual/en/functions.arguments.php

Universidad Privada Domingo Savio. (2026). *Guías y materiales de laboratorio 01–04 de Tecnología Web II* .